UNIVERSIDAD TECNOLÓGICA NACIONAL

Bodegas del Atuel: Informe Final

Índice de contenidos

1. Introducción y alcance **3**
Qué cubre el sistema 3
Alcance 3
Idea central del diseño 3
Tecnologías 3
2. Modelo de datos **4**
Diagrama Entidad-Relación 4
Esquema Relacional 4
Normalización 5
1. Primera Forma Normal (1FN) 5
2. Segunda Forma Normal (2FN) 5
3. Tercera Forma Normal (3FN) 6
4. El caso localidad → encargado regional 8
5. Atributos derivados y 3FN 8
Decisiones de diseño 8
1. Jerarquía de clientes (supertipo / subtipos) 8
2. Claves naturales vs. subrogadas 9
3. Tipos de datos 10
4. Atributos derivados: almacenados vs. calculados 11
5. Acciones referenciales (ON DELETE / ON UPDATE) 12
6. Otras decisiones relevantes 14
Diccionario de datos 14
3. Integridad y lógica de la base **28**
4. Carga masiva **31**
5. Índices y rendimiento **33**
Tabla 1. Índices creados (008_indices.sql) 33
Tabla 2. Comparativa antes / después 34
Tabla 3. Índices evaluados y descartados 34
Tabla 4. Costo de escritura 35
6. Consultas SQL **35**
7. API y front **37**
Arquitectura 37
Acceso a datos y seguridad 37
Endpoints 37
Traducción de errores 38
Front 39

Pruebas automáticas 39
Demostración de inyección SQL fallida 40
8. Despliegue en la nube **40**
Elección de proveedores 40
Configuración de seguridad 40
Migraciones y datos 41
Backup y restauración 41
Una regla de la base funcionando en la nube 41
Limitaciones del plan gratuito 41
9. Conclusiones **42**

1. Introducción y alcance

Bodegas del Atuel es una empresa vitivinícola ficticia que integra la producción de uva y de vino con su distribución y venta a través de una red de salas de venta distribuidas en distintas localidades.

El objetivo de este trabajo integrador es modelar el escenario, implementarlo en PostgreSQL, protegerlo con reglas de integridad escritas en la propia base, cargarlo con un volumen de datos realista, optimizarlo con índices medidos y publicarlo en la nube.

Qué cubre el sistema

a. Producción: fincas, parcelas plantadas con una cepa, lotes de cosecha por campaña y análisis de calidad.
b. Productos: vinos con su bodega, composición por cepas (corte) y maridajes.
c. Logística: ingresos de vino al depósito con un transportista y transferencias del depósito a las salas, cada uno con su ciclo de estados (en confección -> terminado -> procesado).
d. Ventas: clientes socios, distribuidores y exportadores, comprobantes con renglones, descuentos por tipo de cliente y control de stock por sala.

Alcance

32 tablas, 8 migraciones, 15 reglas de negocio, más de 100.000 filas, 5 índices con evidencia de plan de ejecución, API REST y front en React desplegados en Render con Neon como base.

Idea central del diseño

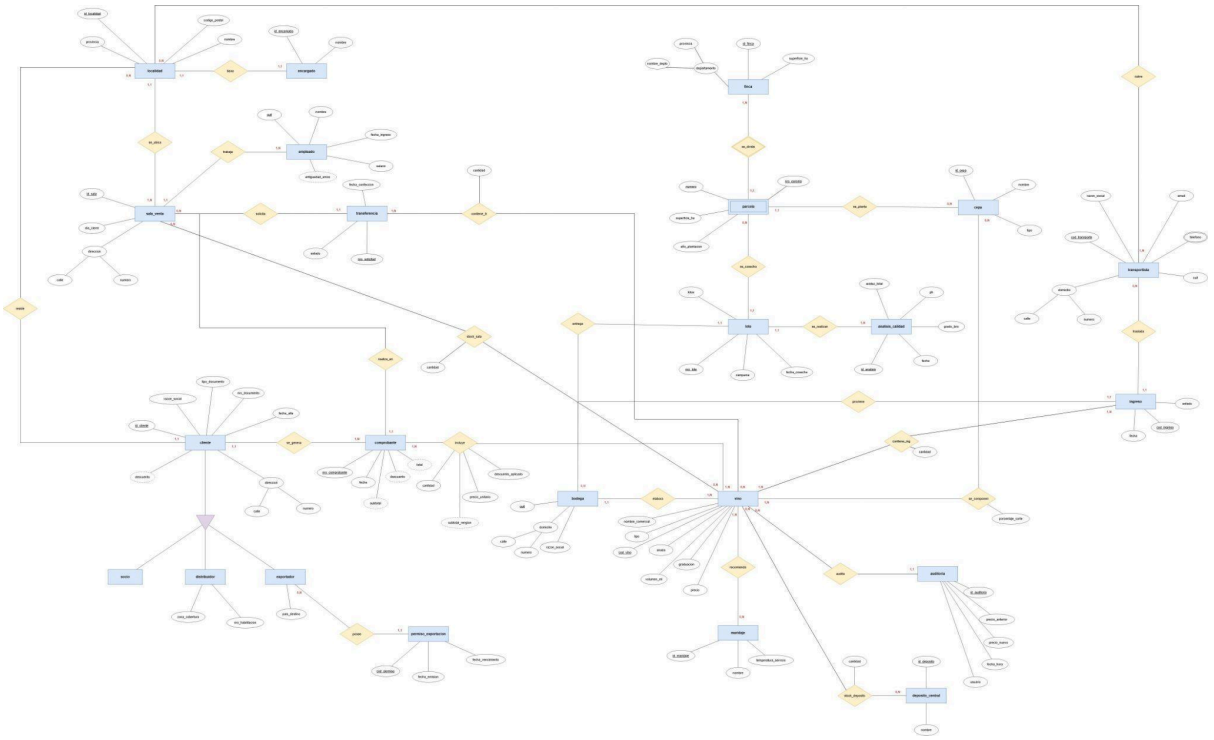
Las reglas de negocio viven en la base. La API y el front no calculan totales, descuentos ni stock: los piden a la base y muestran lo que esta devuelve.

Tecnologías

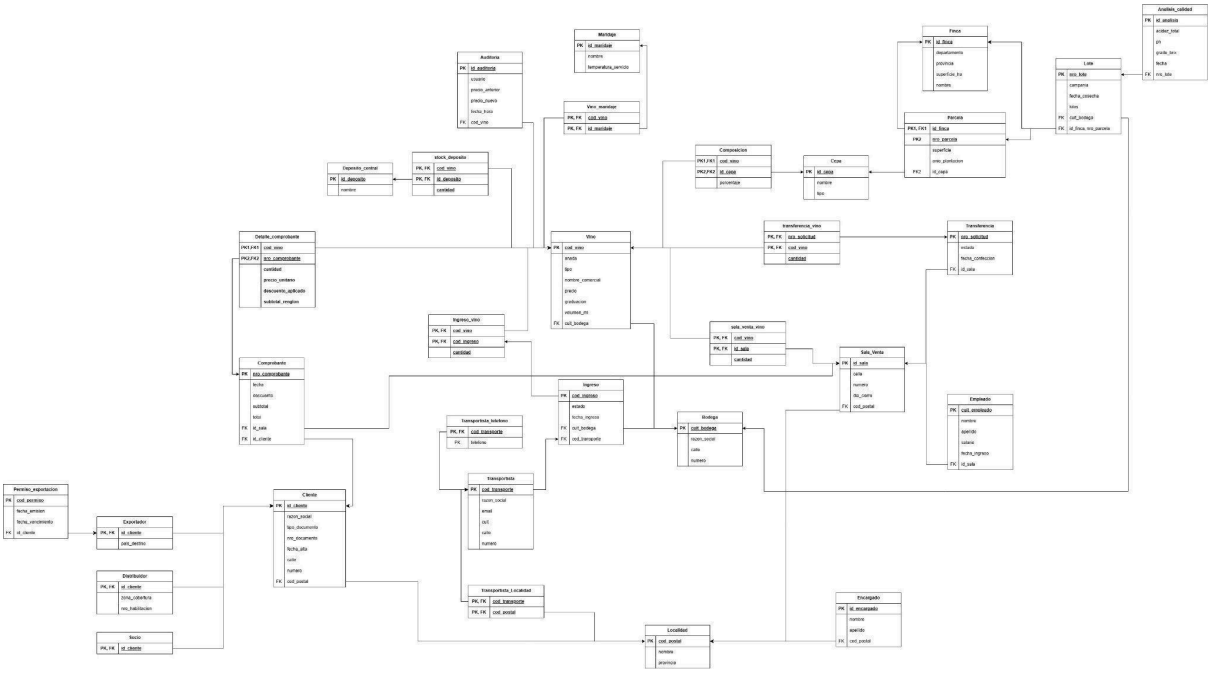
PostgreSQL 16 (local, en Docker) y 18 (Neon), Python con FastAPI y psycopg 3, React con Vite, Docker Compose, Render.

2. Modelo de datos

Diagrama Entidad-Relación



Esquema Relacional



Normalización

1. Primera Forma Normal (1FN)

Una tabla está en 1FN si todos sus atributos son atómicos y no hay grupos repetidos. El esquema la cumple por tres decisiones tomadas al pasar del DER al relacional:

Problema potencial	Solución adoptada
Atributo multivaluado: un transportista tiene varios teléfonos	Tabla Transportista_Telefono con PK (cod_transporte, telefono)
Atributo compuesto: domicilio	Se descompone en calle y numero (Bodega, Sala_Venta, Cliente, Transportista)
Grupos repetidos: los vinos de un comprobante, de un ingreso o de una transferencia	No son columnas repetidas en la cabecera sino filas en Detalle_Comprobante, Ingreso_Vino y Transferencia_Vino. Lo mismo para composición de cepas (Composicion), maridajes (Vino_Maridaje), stock por sala (Sala_Venta_Vino) y stock del depósito (Stock_Deposito)

También se resolvió la relación N:M entre transportistas y localidades con Transportista_Localidad.

2. Segunda Forma Normal (2FN)

Una tabla en 1FN está en 2FN si ningún atributo no clave depende solo de una parte de la clave. Solo puede fallar en tablas con **clave compuesta**; las que tienen clave simple cumplen 2FN automáticamente.

Clave compuesta	Atributos no clave	Por qué depende de la clave completa
(id_finca, nro_parcela) — Parcela	superficie, anio_plantacion, id_cepa	El número de parcela solo identifica a la parcela *dentro de una finca* (entidad débil).
(cod_vino, id_cepa) — Composicion	porcentaje	Es el porcentaje de *esa* cepa en *ese* vino; no de la cepa ni del vino por separado.
(nro_comprobante, cod_vino) — Detalle_Comprobante	cantidad, precio_unitario, descuento_aplicado, subtotal_renglon	precio_unitario es el precio *al momento de esa venta* (no el precio actual del vino); depende de las dos columnas.
(cod_ingreso, cod_vino) — Ingreso_Vino	cantidad	Botellas de ese vino en ese ingreso.
(nro_solicitud, cod_vino) — Transferencia_Vino	cantidad	Botellas de ese vino en esa transferencia.
(id_sala, cod_vino) — Sala_Venta_Vino	cantidad	Stock de ese vino en esa sala.
(id_deposito, cod_vino) — Stock_Deposito	cantidad	Stock de ese vino en el depósito.

Vino_Maridaje, Transportista_Telefono y Transportista_Localidad tienen clave compuesta pero **ningún atributo no clave**, por lo que cumplen 2FN trivialmente.

Nota sobre Detalle_Comprobante.precio_unitario: guardar el precio en el renglón **no** viola la normalización. Si se tomara Vino.precio al consultar, el historial cambiaría cada vez que cambia el precio. El precio del renglón es un dato histórico propio de la venta (ver sección 2.4).

3. Tercera Forma Normal (3FN)

Una tabla en 2FN está en 3FN si ningún atributo no clave depende de otro atributo no clave (dependencia transitiva). **Las claves candidatas (columnas con UNIQUE) pueden determinar otros atributos sin romper la 3FN**, porque no son atributos "no clave".

Tabla	Dependencias funcionales	3FN
Localidad	`cod_postal → nombre, provincia`	Cumple
Encargado	`id_encargado → nombre, apellido, cod_postal; cod_postal es clave candidata (UNIQUE)`	Cumple
Sala_Venta	`id_sala → calle, numero, dia_cierre, cod_postal`	Cumple (ver 1.4)
Empleado	`cuit_empleado → nombre, apellido, salario, fecha_ingreso, id_sala`	Cumple (la antigüedad es calculada, no se almacena)
Finca	`id_finca → nombre, superficie_ha, departamento, provincia`	Cumple
Cepa, Maridaje, Deposito_Central	La clave determina todo lo demás; nombre / nombre_maridaje son claves candidatas	Cumplen
Parcela	`(id_finca, nro_parcela) → superficie, anio_plantacion, id_cepa`	Cumple
Lote	`nro_lote → campania, fecha_cosecha, kilos, cuit_bodega, id_finca, nro_parcela; (id_finca, nro_parcela, campania) es clave candidata (RN-04)`	Cumple
Analisis_Calidad	`id_analisis → fecha, grado_brix, ph, acidez_total, nro_lote`	Cumple
Bodega	`cuit_bodega → razon_social, calle, numero`	Cumple
Vino	`cod_vino → nombre_comercial, tipo, anada, volumen_ml, graduacion, precio,`	Cumple

Tabla	Dependencias funcionales	3FN
	`cuit_bodega; (nombre_comercial, anada, volumen_ml) es clave candidata`	
Composicion / Vino_Maridaje	La clave compuesta determina porcentaje (o no hay atributos no clave)	Cumplen
Cliente	`id_cliente → todos los atributos; (tipo_documento, nro_documento) es clave candidata`	Cumple
Socio, Distribuidor, Exportador	`id_cliente → atributos del subtipo (incluido porcentaje_descuento)`	Cumplen (ver 2.1)
Permiso_exportacion	`cod_permiso → fecha_emision, fecha_vencimiento, id_cliente`	Cumple
Comprobante	`nro_comprobante → fecha, id_cliente, id_sala, subtotal, descuento, total`	Cumple (derivados controlados, ver 2.4)
Detalle_Comprobante	`(nro_comprobante, cod_vino) → cantidad, precio_unitario, descuento_aplicado, subtotal_renglon`	Cumple (derivado controlado, ver 2.4)
Transportista	`cod_transporte → razon_social, email, cuit, calle, numero; cuit es clave candidata`	Cumple
Ingreso	`cod_ingreso → estado, fecha_ingreso, cuit_bodega, cod_transporte`	Cumple
Transferencia	`nro_solicitud → estado, fecha, id_sala`	Cumple
Auditoria	`id_auditoria → cod_vino, usuario, precio_anterior, precio_nuevo, fecha_hora`	Cumple

4. El caso `localidad → encargado regional`

Esta es la dependencia funcional que más podía romper la 3FN. La regla de negocio (RN-02) dice que **hay un único encargado regional por localidad y está afectado a todas las salas de esa localidad**.

Si se hubiera guardado id_encargado en Sala_Venta, aparecería:

`id_sala → cod_postal → id_encargado` (dependencia transitiva)

es decir, el encargado dependería de la localidad y no de la sala: se repetiría en cada sala de la misma localidad (redundancia) y podría quedar inconsistente (dos salas de una misma localidad con encargados distintos).

Solución adoptada: Sala_Venta solo guarda cod_postal. La relación con el encargado vive en Encargado, que tiene la FK `cod_postal → Localidad` con restricción UNIQUE (unique_localidad), garantizando un encargado por localidad. El encargado de una sala se obtiene por join: `Sala_Venta → Localidad → Encargado`. Así la dependencia `cod_postal → encargado` queda en una única tabla cuya clave candidata es justamente cod_postal, y la 3FN se cumple.

5. Atributos derivados y 3FN

subtotal, descuento y total (Comprobante) y subtotal_renglon (Detalle_Comprobante) son atributos derivados: podrían calcularse a partir de otros datos. Los conservamos almacenados, pero su valor lo fija siempre la base y no puede modificarse de forma independiente, por lo que no generan una dependencia transitiva real (se justifica en la sección Decisiones de diseño, punto 4). La antigüedad del empleado, en cambio, no se almacena: cambia con el tiempo y guardarla la dejaría desactualizada.

Decisiones de diseño

1. Jerarquía de clientes (supertipo / subtipos)

Situación: un cliente puede ser socio, distribuidor o exportador. Cada tipo tiene atributos propios (zona de cobertura y nº de habilitación; país de destino y permisos de exportación) y un porcentaje de descuento distinto (10 / 20 / 30 %, RN-06).

Alternativas evaluadas

Alternativa	Ventaja	Problema
Una sola tabla Cliente con todas las columnas y un campo tipo	Simple, sin joins	Muchos NULL (cada fila usa solo una parte de las columnas); no se pueden marcar como NOT NULL los atributos propios de un tipo
Una tabla por subtipo, sin supertipo	Sin nulos	Se duplican los datos comunes (documento, domicilio) y Comprobante no puede tener una única FK a "cliente"
Supertipo + una tabla por subtipo (elegida)	Datos comunes una sola vez; atributos propios NOT NULL; Comprobante referencia un único id_cliente	Requiere join para conocer los atributos del subtipo

Implementación: Cliente(id_cliente PK, ...) con Socio, Distribuidor y Exportador, cuya PK es a su vez FK a Cliente(id_cliente) con ON DELETE CASCADE (el subtipo no existe sin su cliente).

Permiso_exportacion referencia a Exportador, no a Cliente, de modo que solo un exportador puede tener permisos (RN-07).

Exclusividad: un cliente pertenece a un solo subtipo. La FK compartida por sí sola no lo impide; lo controla fn_descuento_cliente, que lanza una excepción si el cliente figura en más de un subtipo o en ninguno, de modo que no se puede registrar una venta a un cliente inconsistente. (Un constraint trigger que lo valide al insertar el subtipo sería una mejora posible.)

Descuento: cada subtipo guarda su porcentaje_descuento con un DEFAULT (10, 20, 30) y un CHECK entre 0 y 100. La función fn_descuento_cliente(id_cliente) devuelve el porcentaje según el subtipo, y el trigger lo copia a Detalle_Comprobante.descuento_aplicado.

Nombre / razón social: Cliente tiene apellido, nombre y razon_social, todas opcionales, con el CHECK check_nombre_o_razon_social: o hay razón social, o hay apellido **y** nombre. Esto cubre personas físicas y empresas sin forzar datos falsos.

2. Claves naturales vs. subrogadas

Criterio: se usa clave natural solo cuando el dato es un identificador **estable, único y propio del dominio**; en el resto se usa una clave subrogada autoincremental.

Tabla	Clave	Tipo	Justificación
Bodega	cuit_bodega	Natural (cuit_dom)	El CUIT es único, legalmente asignado y no cambia
Empleado	cuit_empleado	Natural (cuit_dom)	Ídem
Localidad	cod_postal	Natural (INT)	Identifica a la localidad en el dominio
Parcela	(id_finca, nro_parcela)	Natural compuesta	Entidad débil: el nº de parcela solo existe dentro de su finca
Tablas N:M (Composicion, Vino_Maridaje, Detalle_Comprobante, Ingreso_Vino, Transferencia_Vino, Stock_Deposito, Sala_Venta_Vino, Transportista_*)	Compuesta por las FKs	Natural compuesta	La combinación identifica la fila; evita una clave extra sin significado
Vino, Cliente, Sala_Venta, Encargado, Finca, Cepa, Maridaje, Lote, Analisis_Calidad, Comprobante, Transportista, Ingreso, Transferencia, Permiso_exportacio	id_* / cod_* / nro_*	Subrogada (IDENTITY o SERIAL)	No hay un identificador natural estable (ej.: dos vinos pueden compartir nombre; un cliente puede identificarse con DNI, CUIT, CUIL o pasaporte)

Tabla	Clave	Tipo	Justificación
n, Deposito_Central, Auditoria			

Claves candidatas que no son PK se protegen con UNIQUE: Vino(nombre_comercial, anada, volumen_ml), Cliente(tipo_documento, nro_documento), Lote(id_finca, nro_parcela, campania), Transportista.cuit, Distribuidor.nro_habilitacion, Cepa.nombre, Maridaje.nombre_maridaje, Encargado.cod_postal.

Por qué subrogada en Cliente: el documento puede ser de cuatro tipos distintos y podría corregirse; usarlo como PK obligaría a propagar el cambio a todas las FKs. Con id_cliente, el documento se mantiene como clave candidata UNIQUE.

3. Tipos de datos

Dato	Tipo elegido	Justificación
Importes (precio, subtotal, total, descuento, salario)	NUMERIC(p,s), p. ej. NUMERIC(10,2) / NUMERIC(12,2)	Precisión exacta. **Nunca FLOAT/REAL**: el punto flotante binario introduce errores de redondeo inaceptables en dinero
Porcentajes, graduación, pH, °Brix, superficie, kilos	NUMERIC con escala acotada	Valores decimales exactos con rango acotado por CHECK
Cantidades de botellas	INT con CHECK (> 0) o (>= 0) en stock	Son enteros; el stock puede ser 0 pero nunca negativo
Fechas de negocio (venta, cosecha, ingreso, emisión)	DATE	Solo interesa el día
Momento de un cambio de precio (Auditoria.fecha_hora)	TIMESTAMP	Se necesita hora para ordenar cambios dentro del mismo día
Año de añada / plantación / campaña	SMALLINT	Rango pequeño, ocupa menos espacio; acotado por CHECK
CUIT	DOMAIN cuit_dom AS CHAR(11) con CHECK (VALUE ~ '^[0-9]{11}$')	Longitud fija; el dominio reutiliza la validación en Bodega, Empleado y Transportista
Estado de ingreso/transferencia	ENUM estado_documento ('en_confeccion','terminado','procesado')	Conjunto cerrado y ordenado de valores; el motor rechaza cualquier otro. Las transiciones se controlan con trigger (RN-10 / RN-11)
Tipo de vino, tipo de cepa, tipo de documento, día de cierre, volumen	VARCHAR + CHECK ... IN (...)	Listas cortas y verificables; más fáciles de cambiar que un ENUM (que requiere ALTER TYPE)
Documento, teléfono, nº de habilitación	VARCHAR	No se operan aritméticamente y pueden tener ceros a la izquierda o formato

Dato	Tipo elegido	Justificación
Claves subrogadas	INT GENERATED ALWAYS AS IDENTITY / SERIAL	Autoincrementales, simples de indexar

Por qué ENUM solo para estados: es el único atributo con un ciclo de vida (en_confección → terminado → procesado) compartido por dos tablas, por lo que conviene un tipo único. Para el resto de las listas cerradas se prefirió CHECK, que se modifica con un simple ALTER TABLE.

4. Atributos derivados: almacenados vs. calculados

Atributo	Decisión	Cómo se garantiza la consistencia
Detalle_Comprobante.subtotal_renglon	**Almacenado**	Lo calcula el trigger (RN-08): cantidad × precio_unitario × (1 - descuento_aplicado/100). CHECK (>= 0)
Comprobante.subtotal	**Almacenado**	El comprobante nace con importes en 0 (trg_comprobante_inicial); trg_detalle_aplicar recalcula la suma de cantidad × precio_unitario de los renglones cada vez que se inserta uno
Comprobante.descuento	**Almacenado**	Importe descontado según el tipo de cliente; lo calcula el trigger. CHECK (descuento >= 0 AND descuento <= subtotal)
Comprobante.total	**Almacenado**	CHECK (total = subtotal - descuento) impide que se guarde un total incoherente; fn_total_comprobante lo calcula
Detalle_Comprobante.precio_unitario	**Almacenado (dato histórico)**	Se copia de Vino.precio al momento de la venta; el cambio posterior del precio no altera ventas pasadas. Los cambios de precio se auditan (RN-13)
Detalle_Comprobante.descuento_aplicado	**Almacenado (dato histórico)**	Porcentaje vigente al momento de la venta, obtenido con fn_descuento_cliente
Antigüedad del empleado	**Calculado, no almacenado**	fn_antiguedad_anios(fecha_ingreso); cambia con el tiempo
Stock de la sala	Almacenado en Sala_Venta_Vino	Lo modifican los triggers de venta y el procedimiento de transferencia; CHECK

Atributo	Decisión	Cómo se garantiza la consistencia
		(cantidad >= 0) y validación previa (RN-09)

Por qué almacenar los importes del comprobante:

1. **Rendimiento:** las consultas de ventas por sala y trimestre (etapa 4) leen total directamente, sin recalcular sobre 600 mil renglones.
2. **Inmutabilidad histórica:** un comprobante emitido es un documento; sus importes no deben cambiar si cambian precios o descuentos futuros.
3. **Costo:** el sobrecosto de mantenerlos (triggers) es aceptable porque las ventas se insertan una vez y no se modifican.

Cómo se evita la inconsistencia: el cálculo no queda en manos de la aplicación; lo hacen triggers y funciones de la base (trg_comprobante_inicial, trg_detalle_validar, trg_detalle_aplicar), y los CHECK bloquean cualquier combinación incoherente (`total = subtotal - descuento`, descuento <= subtotal). Las pruebas de tests/db_tests.sql verifican RN-08.

5. Acciones referenciales (ON DELETE / ON UPDATE)

Criterio general

- **ON DELETE RESTRICT** (o NO ACTION) en las entidades de negocio y en los movimientos: no se puede borrar algo que otra tabla usa. Protege el historial (RN-14).
- **ON DELETE CASCADE** solo en tablas **dependientes que no tienen sentido sin su tabla principal**: renglones, teléfonos, composición, vino-maridaje, permisos, análisis, parcelas y subtipos de cliente.
- **ON UPDATE CASCADE** solo en FKs hacia **claves naturales** (cuit_bodega, cod_postal), que podrían corregirse. Las claves IDENTITY no se modifican, así que no necesitan ON UPDATE.

Resumen de las 37 FK del esquema (21 RESTRICT, 16 CASCADE)

Acción	FKs	Razón
ON DELETE CASCADE	`Parcela→Finca; Analisis_Calidad→Lote; Vino_Maridaje→Vino y →Maridaje; Detalle_Comprobante→Comprobante; Socio / Distribuidor / Exportador→Cliente; Permiso_exportacion→Exportador; Ingreso_Vino→Ingreso; Transferencia_Vino→Transferencia; Transportista_Telefono→Transportista; Transportista_Localidad→Transportista y`	La fila hija es parte del padre (detalle, subtipo, atributo multivaluado, asociación N:M)

Acción	FKs	Razón
	`->Localidad; Stock_Deposito->Deposito_Central; Composicion->Vino`	
ON DELETE RESTRICT	`Vino->Bodega; Encargado / Sala_Venta / Cliente->Localidad; Empleado->Sala_Venta; Parcela->Cepa; Lote->Bodega y ->Parcela; Comprobante->Sala_Venta y ->Cliente; Detalle_Comprobante->Vino ; Ingreso->Bodega y ->Transportista; Ingreso_Vino->Vino; Stock_Deposito->Vino; Transferencia->Sala_Venta ; Transferencia_Vino->Vino; Sala_Venta_Vino->Sala_Venta y ->Vino; Auditoria->Vino; Composicion->Cepa`	Son entidades de negocio o movimientos con valor histórico; no se pueden borrar si están referenciadas
ON UPDATE CASCADE	`Vino->Bodega; Encargado->Localidad; Sala_Venta->Localidad; Cliente->Localidad`	Se apunta a claves naturales (cuit_bodega, cod_postal)

Ejemplos que justifican la elección

- No se puede borrar un Vino con ventas, ingresos, transferencias o stock (RESTRICT), pero sí se pueden borrar sus filas de composición y maridajes (CASCADE), que solo lo describen.
- Borrar un Comprobante borra sus renglones (CASCADE), pero no se puede borrar un Cliente o una Sala_Venta con comprobantes (RESTRICT).
- Un Ingreso o Transferencia ya procesado no se puede alterar: lo impiden los triggers de estado (RN-10 a RN-12), que se suman a las FK.
- Borrar una Finca borra sus parcelas (entidad débil), pero la parcela no puede borrarse si ya tiene lotes cosechados (`Lote->Parcela RESTRICT`).

6. Otras decisiones relevantes

- **Entidad débil Parcela:** PK compuesta (id_finca, nro_parcela). Lote referencia la parcela con una FK compuesta; el UNIQUE (id_finca, nro_parcela, campania) garantiza a lo sumo una cosecha por parcela y campaña (RN-04), y CHECK (campania = EXTRACT(YEAR FROM fecha_cosecha)) mantiene coherentes año y fecha.
- **Una única cepa por parcela (RN-04):** id_cepa en Parcela (relación 1:N), no una tabla intermedia.

- **Composición de cada vino:** tabla N:M con porcentaje; la suma por vino debe ser 100 y se controla con un **constraint trigger diferido** (trg_composicion_100), que valida al final de la transacción y permite insertar las cepas una por una.
- **Precio único para toda la red (RN-01):** precio vive en Vino, no en cada sala.
- **Stock separado por ubicación:** Stock_Deposito y Sala_Venta_Vino son tablas distintas con clave (ubicación, vino); evitan columnas por sala y permiten agregar salas sin cambiar el esquema.
- **Un único depósito central:** se modela como tabla Deposito_Central (y no como constante) para poder referenciarlo con FK y mantener el stock normalizado.
- **Auditoría de precios (RN-13):** tabla Auditoria completada por un trigger AFTER UPDATE sobre Vino, de modo que no se puede omitir desde la aplicación.
- **Integridad por capas:** tipos y DOMAIN → CHECK → FK → triggers → procedimientos. Las reglas que involucran varias filas o tablas (stock, estados, suma 100 %) se resuelven con triggers; las de una sola fila con CHECK.

Diccionario de datos

Convenciones usadas en las tablas

Tipos propios: estado_documento es un ENUM con los valores en_confeccion, terminado y procesado. cuit_dom es un DOMAIN basado en CHAR(11) que admite solo 11 dígitos. Los importes usan NUMERIC (nunca FLOAT).

Claves: se usan claves naturales para el CUIT (bodega, empleado) y el código postal (localidad), porque son identificadores estables y únicos del dominio. El resto de las tablas usa claves subrogadas autoincrementales (IDENTITY).

Acciones referenciales: ON DELETE RESTRICT en las entidades de negocio y en los movimientos (RN-14). ON DELETE CASCADE solo en tablas dependientes que no existen sin su tabla principal (renglones, teléfonos, composición, vino-maridaje, permisos, análisis, parcelas y subtipos de cliente). ON UPDATE CASCADE solo en claves foráneas hacia claves naturales; las claves IDENTITY no se modifican.

Atributos derivados: subtotal, descuento y total del comprobante y subtotal_renglon del detalle se almacenan, pero los calcula la base (RN-08). La antigüedad del empleado no se almacena: se calcula con fn_antiguedad_anios.

Columna Nulo: No equivale a NOT NULL. Las claves primarias nunca admiten nulos.

Tabla: Localidad

Localidades donde la empresa tiene salas de venta y donde viven los clientes. Identificadas por código postal.

Campo	Tipo	Clave	Nulo	Referencia	Descripción	Restricciones
cod_postal	VARCHAR(20)	PK	No		Código postal de la localidad	
nombre	VARCHAR(50)		No		Nombre de la localidad	

Campo	Tipo	Clave	Nulo	Referencia	Descripción	Restricciones
provincia	VARCHAR(50)		No		Provincia a la que pertenece	

Tabla: Encargado

Encargado regional (responsable comercial). Hay uno solo por localidad y está afectado a todas las salas de esa localidad (RN-02).

Campo	Tipo	Clave	Nulo	Referencia	Descripción	Restricciones
id_encargado	INT	PK	No		ID único del encargado	Autoincremental (IDENTITY)
nombre	VARCHAR(50)		No		Nombre del encargado	
apellido	VARCHAR(50)		No		Apellido del encargado	
cod_postal	VARCHAR(20)	FK	No	Localidad.cod_postal ON DELETE RESTRICT ON UPDATE CASCADE	Localidad de la que es responsable	UNIQUE (un solo encargado por localidad, RN-02)

Tabla: Sala_Venta

Salas de venta de la red. Cada sala está en una sola localidad; una localidad puede tener varias salas.

Campo	Tipo	Clave	Nulo	Referencia	Descripción	Restricciones
id_sala	INT	PK	No		ID único de la sala de venta	Autoincremental (IDENTITY)
calle	VARCHAR(50)		No		Domicilio de la sala: calle	
numero	INT		No		Domicilio de la sala: numeración	CHECK (numero > 0)
dia_cierre	VARCHAR(20)		No		Día de cierre semanal de la sala	CHECK (dia_cierre IN ('lunes','martes','miercoles','jueves','viernes','sabado','domingo'))
cod_postal	VARCHAR(20)	FK	No	Localidad.cod_postal ON DELETE RESTRICT ON UPDATE CASCADE	Localidad donde está la sala (RN-02)	

Tabla: Empleado

Empleados de la empresa. Cada empleado trabaja en una sola sala (RN-03). La antigüedad no se almacena: se calcula con fn_antiguedad_anios.

Campo	Tipo	Clave	Nulo	Referencia	Descripción	Restricciones
cuit_empleado	cuit_dom	PK	No		CUIT del empleado	Dominio cuit_dom (11 dígitos)
nombre	VARCHAR(50)		No		Nombre del empleado	
apellido	VARCHAR(50)		No		Apellido del empleado	
salario	NUMERIC(12, 2)		No		Salario del empleado	CHECK (salario > 0) (RN-15)
fecha_ingreso	DATE		No		Fecha de ingreso laboral	
id_sala	INT	FK	No	Sala_Venta.id_sala ON DELETE RESTRICT	Sala donde trabaja (RN-03)	

Tabla: Finca

Fincas donde la empresa cultiva uva. Se divide en parcelas.

Campo	Tipo	Clave	Nulo	Referencia	Descripción	Restricciones
id_finca	INT	PK	No		ID único de la finca	Autoincremental (IDENTITY)
nombre	VARCHAR(50)		No		Nombre de la finca	
superficie_ha	NUMERIC(8,2)		No		Superficie total en hectáreas	CHECK (superficie_ha > 0) (RN-15)
departamento	VARCHAR(50)		No		Departamento donde se ubica	
provincia	VARCHAR(50)		No		Provincia donde se ubica	

Tabla: Cepa

Catálogo de cepas (variedades de uva).

Campo	Tipo	Clave	Nulo	Referencia	Descripción	Restricciones
id_cepa	INT	PK	No		ID único de la cepa	Autoincremental (IDENTITY)
nombre	VARCHAR(50)		No		Nombre de la cepa (ej. Malbec)	UNIQUE
tipo	VARCHAR(10)		No		Tipo de cepa: tinta o blanca	CHECK (tipo IN ('tinta','blanca'))

Tabla: Parcela

Subdivisión de una finca. Entidad débil: se identifica por (id_finca, nro_parcela) y no existe sin su finca. Tiene una sola cepa plantada (RN-04).

Campo	Tipo	Clave	Nulo	Referencia	Descripción	Restricciones
id_finca	INT	PK, FK	No	Finca.id_finca ON DELETE CASCADE	Finca a la que pertenece la parcela	
nro_parcela	INT	PK	No		Número de parcela dentro de la finca	
superficie	NUMERIC(8,2)		No		Superficie de la parcela en hectáreas	CHECK (superficie > 0) (RN-15)
anio_plantacion	SMALLINT		No		Año de plantación	CHECK (anio_plantacion BETWEEN 1900 AND 2100)
id_cepa	INT	FK	No	Cepa.id_cepa ON DELETE RESTRICT	Cepa plantada (una sola, RN-04)	

Tabla: Bodega

Bodegas del grupo donde se elabora el vino.

Campo	Tipo	Clave	Nulo	Referencia	Descripción	Restricciones
cuit_bodega	cuit_dom	PK	No		CUIT de la bodega (clave natural)	Dominio cuit_dom (11 dígitos)
razon_social	VARCHAR(100)		No		Nombre legal de la bodega	
calle	VARCHAR(50)		No		Domicilio de la bodega: calle	
numero	INT		No		Domicilio de la bodega: numeración	CHECK (numero > 0)

Tabla: Lote

Lote de uva generado al cosechar una parcela en una campaña. Una parcela se cosecha a lo sumo una vez por campaña (RN-04).

Campo	Tipo	Clave	Nulo	Referencia	Descripción	Restricciones
nro_lote	INT	PK	No		Número único del lote	Autoincremental (IDENTITY)

Campo	Tipo	Clave	Nulo	Referencia	Descripción	Restricciones
campania	SMALLINT		No		Campaña (año de cosecha)	UNIQUE (id_finca, nro_parcela, campania) (RN-04) CHECK (campania = EXTRACT(YEAR FROM fecha_cosecha))
fecha_cosecha	DATE		No		Fecha de cosecha	
kilos	NUMERIC(10, 2)		No		Kilos cosechados	CHECK (kilos > 0) (RN-15)
cuit_bodega	cuit_dom	FK	No	Bodega.cuit_bodega ON DELETE RESTRICT ON UPDATE CASCADE	Bodega a la que fue entregado el lote	
id_finca	INT	FK	No	Parcela(id_finca, nro_parcela) ON DELETE RESTRICT	Finca de la parcela cosechada (FK compuesta, una sola con nro_parcela)	Parte del UNIQUE
nro_parcela	INT	FK	No	(parte de la FK compuesta a Parcela)	Parcela cosechada	Parte del UNIQUE

Tabla: Analisis_Calidad

Análisis de calidad practicados a un lote de uva (uno o más por lote).

Campo	Tipo	Clave	Nulo	Referencia	Descripción	Restricciones
id_analisis	INT	PK	No		ID único del análisis	Autoincremental (IDENTITY)
fecha	DATE		No		Fecha del análisis	
grado_brix	NUMERIC(4,2)		No		Grado Brix (azúcares) de la uva	CHECK (grado_brix > 0)
ph	NUMERIC(4,2)		No		pH del mosto	CHECK (ph BETWEEN 0 AND 14)
acidez_total	NUMERIC(5,2)		No		Acidez total (g/L)	CHECK (acidez_total > 0)
nro_lote	INT	FK	No	Lote.nro_lote ON DELETE CASCADE	Lote analizado	

Tabla: Vino

Vinos elaborados por las bodegas del grupo. El precio es único para toda la red de salas (RN-01).

Campo	Tipo	Clave	Nulo	Referencia	Descripción	Restricciones
cod_vino	INT	PK	No		Código único del vino	Autoincremental (IDENTITY)
anada	SMALLINT		No		Año de cosecha (añada)	CHECK (anada BETWEEN 1990 AND 2100) UNIQUE (nombre_comercial, anada, volumen_ml)
tipo	VARCHAR(12)		No		Tinto, blanco, rosado o espumante	CHECK (tipo IN ('tinto','blanco','rosado','espumante'))
nombre_comercial	VARCHAR(80)		No		Nombre comercial de venta	Parte del UNIQUE (nombre_comercial, anada, volumen_ml)
precio	NUMERIC(10,2)		No		Precio de venta, igual para todas las salas	CHECK (precio > 0) (RN-01, RN-15) Cada cambio se audita (RN-13)
graduacion	NUMERIC(4,2)		No		Graduación alcohólica (°)	CHECK (graduacion BETWEEN 5 AND 20) (RN-15)
volumen_ml	INT		No		Presentación en mililitros	CHECK (volumen_ml IN (187,375,750,1500,3000)) Parte del UNIQUE
cuit_bodega	cuit_dom	FK	No	Bodega.cuit_bodega ON DELETE RESTRICT ON UPDATE CASCADE	Bodega que elabora el vino (RN-14)	

Tabla: Composición

Cepas que componen cada vino y su porcentaje de corte (relación N:M con atributo).

Campo	Tipo	Clave	Nulo	Referencia	Descripción	Restricciones
cod_vino	INT	PK, FK	No	Vino.cod_vino ON DELETE CASCADE	Vino	
id_cepa	INT	PK, FK	No	Cepa.id_cepa ON DELETE RESTRICT	Cepa utilizada	

Campo	Tipo	Clave	Nulo	Referencia	Descripción	Restricciones
porcentaje	NUMERIC(5,2)		No		Porcentaje de la cepa en el corte	CHECK (porcentaje > 0 AND porcentaje <= 100) La suma por vino debe ser 100 (constraint trigger diferido, RN-05)

Tabla: Maridaje

Catálogo de maridajes recomendados.

Campo	Tipo	Clave	Nulo	Referencia	Descripción	Restricciones
id_maridaje	INT	PK	No		ID único del maridaje	Autoincremental (IDENTITY)
nombre	VARCHAR(50)		No		Nombre del maridaje (ej. carnes rojas)	UNIQUE
temperatura_servicio	NUMERIC(4,2)		No		Temperatura de servicio sugerida en °C	CHECK (temperatura_servicio BETWEEN 0 AND 30)

Tabla: Vino_maridaje

Maridajes recomendados para cada vino.

Campo	Tipo	Clave	Nulo	Referencia	Descripción	Restricciones
cod_vino	INT	PK, FK	No	Vino.cod_vino ON DELETE CASCADE	Vino	
id_maridaje	INT	PK, FK	No	Maridaje.id_maridaje ON DELETE RESTRICT	Maridaje recomendado	

Tabla: Cliente

Supertipo de la jerarquía de clientes (socio, distribuidor, exportador).

Campo	Tipo	Clave	Nulo	Referencia	Descripción	Restricciones
id_cliente	INT	PK	No		ID único del cliente	Autoincremental (IDENTITY)
razon_social	VARCHAR(100)		No		Apellido y nombre, o razón social, del cliente	

Campo	Tipo	Clave	Nulo	Referencia	Descripción	Restricciones
tipo_documento	VARCHAR(10)		No		Tipo de documento	CHECK (tipo_documento IN ('DNI','CUIT','CUIL','PASAPORTE')) Parte del UNIQUE (tipo_documento, nro_documento)
nro_documento	VARCHAR(20)		No		Número de documento	Parte del UNIQUE (tipo_documento, nro_documento)
fecha_alta	DATE		No		Fecha de alta del cliente	DEFAULT CURRENT_DATE
calle	VARCHAR(50)		No		Domicilio: calle	
numero	INT		No		Domicilio: numeración	CHECK (numero > 0)
cod_postal	VARCHAR(20)	FK	No	Localidad.cod_postal ON DELETE RESTRICT ON UPDATE CASCADE	Localidad del cliente	

Tabla: Socio

Subtipo de Cliente: socio del club (consumidor final), 10 % de descuento (RN-06).

Campo	Tipo	Clave	Nulo	Referencia	Descripción	Restricciones
id_cliente	INT	PK, FK	No	Cliente.id_cliente ON DELETE CASCADE	Cliente que es socio	Un cliente pertenece a un solo subtipo

Tabla: Distribuidor

Subtipo de Cliente: distribuidor, 20 % de descuento (RN-06).

Campo	Tipo	Clave	Nulo	Referencia	Descripción	Restricciones
id_cliente	INT	PK, FK	No	Cliente.id_cliente ON DELETE CASCADE	Cliente que es distribuidor	Un cliente pertenece a un solo subtipo
zona_cobertura	VARCHAR(50)		No		Zona de cobertura del distribuidor	
nro_habilitacion	VARCHAR(20)		No		Número de habilitación	UNIQUE

Tabla: Exportador

Subtipo de Cliente: exportador, 30 % de descuento (RN-06). Puede tener varios permisos de exportación.

Campo	Tipo	Clave	Nulo	Referencia	Descripción	Restricciones
id_cliente	INT	PK, FK	No	Cliente.id_cliente ON DELETE CASCADE	Cliente que es exportador	Un cliente pertenece a un solo subtipo
pais_destino	VARCHAR(50)		No		País de destino principal	

Tabla: Permiso_exportacion

Permisos de exportación de un exportador (RN-07).

Campo	Tipo	Clave	Nulo	Referencia	Descripción	Restricciones
cod_permiso	INT	PK	No		Código del permiso	Autoincremental (IDENTITY)
fecha_emision	DATE		No		Fecha de emisión	
fecha_vencimiento	DATE		No		Fecha de vencimiento	CHECK (fecha_vencimiento > fecha_emision) (RN-07)
id_cliente	INT	FK	No	Exportador.id_cliente ON DELETE CASCADE	Exportador titular del permiso	

Tabla: Comprobante

Cabecera de cada venta. Subtotal, descuento y total son derivados: se almacenan, pero los calcula la base (trigger y fn_total_comprobante).

Campo	Tipo	Clave	Nulo	Referencia	Descripción	Restricciones
nro_comprobante	INT	PK	No		Número del comprobante	Autoincremental (IDENTITY)
fecha	DATE		No		Fecha de la venta	DEFAULT CURRENT_DATE
subtotal	NUMERIC(12, 2)		No		Suma de cantidad x precio unitario de los renglones, sin descuento (derivado)	DEFAULT 0 CHECK (subtotal >= 0) Lo calcula el trigger (RN-08)

Campo	Tipo	Clave	Nulo	Referencia	Descripción	Restricciones
descuento	NUMERIC(12, 2)		No		Importe descontado según el tipo de cliente (derivado)	DEFAULT 0 CHECK (descuento >= 0 AND descuento <= subtotal) Lo calcula el trigger (RN-08)
total	NUMERIC(12, 2)		No		Total a pagar: subtotal menos descuento (derivado)	DEFAULT 0 CHECK (total = subtotal - descuento) Lo calcula el trigger (RN-08)
id_sala	INT	FK	No	Sala_Venta.id_sala ON DELETE RESTRICT	Sala donde se realizó la venta	
id_cliente	INT	FK	No	Cliente.id_cliente ON DELETE RESTRICT	Cliente que compró	

Tabla: Detalle_comprobante

Renglones de un comprobante (relación N:M con atributos entre Comprobante y Vino).

Campo	Tipo	Clave	Nulo	Referencia	Descripción	Restricciones
cod_vino	INT	PK, FK	No	Vino.cod_vino ON DELETE RESTRICT	Vino vendido	Trigger BEFORE INSERT valida stock de la sala (RN-09)
nro_comprobante	INT	PK, FK	No	Comprobante.nro_comprobante ON DELETE CASCADE	Comprobante al que pertenece el renglón	
cantidad	INT		No		Cantidad de botellas vendidas	CHECK (cantidad > 0) (RN-15)
precio_unitario	NUMERIC(10,2)		No		Precio del vino al momento de la venta	CHECK (precio_unitario > 0) (RN-15)
descuento_aplicado	NUMERIC(5,2)		No		Porcentaje de descuento según el tipo de cliente (10, 20 o 30 %), obtenido con fn_descuento_cliente	CHECK (descuento_aplicado BETWEEN 0 AND 100) (RN-06)

Campo	Tipo	Clave	Nulo	Referencia	Descripción	Restricciones
subtotal_renglon	NUMERIC(12, 2)		No		Importe del renglón con descuento: cantidad x precio_unitario x (1 - descuento_aplicado/100) (derivado)	CHECK (subtotal_renglon >= 0) Lo calcula el trigger (RN-08)

Tabla: Transportista

Empresas de transporte que trasladan los ingresos al depósito central.

Campo	Tipo	Clave	Nulo	Referencia	Descripción	Restricciones
cod_transporte	INT	PK	No		Código de la empresa de transporte	Autoincremental (IDENTITY)
razon_social	VARCHAR(100)		No		Razón social de la empresa	
email	VARCHAR(100)		No		Correo electrónico de contacto	CHECK (email LIKE '%_@_%._%')
cuit	cuit_dom		No		CUIT de la empresa (clave candidata)	UNIQUE Dominio cuit_dom (11 dígitos)
calle	VARCHAR(50)		No		Domicilio: calle	
numero	INT		No		Domicilio: numeración	CHECK (numero > 0)

Tabla: Transportista_telefono

Teléfonos de cada transportista (atributo multivaluado, 1FN).

Campo	Tipo	Clave	Nulo	Referencia	Descripción	Restricciones
cod_transporte	INT	PK, FK	No	Transportista.cod_transporte ON DELETE CASCADE	Transportista	
telefono	VARCHAR(20)	PK	No		Número de teléfono	

Tabla: Transportista_Localidad

Localidades a las que llega el servicio de cada transportista (relación N:M).

Campo	Tipo	Clave	Nulo	Referencia	Descripción	Restricciones
cod_transporte	INT	PK, FK	No	Transportista.cod_transporte ON DELETE CASCADE	Transportista	
cod_postal	VARCHAR(20)	PK, FK	No	Localidad.cod_postal ON DELETE RESTRICT ON UPDATE CASCADE	Localidad cubierta por el servicio	

Tabla: Ingreso

Ingreso de vino al depósito central. Estados: en_confeccion → terminado → procesado, sin retrocesos (RN-10). Solo uno terminado impacta en el stock.

Campo	Tipo	Clave	Nulo	Referencia	Descripción	Restricciones
cod_ingreso	INT	PK	No		Código único de ingreso	Autoincremental (IDENTITY)
estado	estado_documento		No		Estado del ingreso: en_confeccion, terminado o procesado	DEFAULT 'en_confeccion' Transiciones controladas por trigger (RN-10)
fecha_ingreso	DATE		No		Fecha del ingreso	DEFAULT CURRENT_DATE
cuit_bodega	cuit_dom	FK	No	Bodega.cuit_bodega ON DELETE RESTRICT ON UPDATE CASCADE	Bodega de origen (RN-14)	
cod_transporte	INT	FK	No	Transportista.cod_transporte ON DELETE RESTRICT	Transportista que efectuó el traslado (RN-14)	

Tabla: Ingreso_vino

Renglones de un ingreso: vinos y cantidad de botellas. Solo se modifican si el ingreso está en confección (RN-12).

Campo	Tipo	Clave	Nulo	Referencia	Descripción	Restricciones
cod_vino	INT	PK, FK	No	Vino.cod_vino ON DELETE RESTRICT	Vino ingresado	
cod_ingreso	INT	PK, FK	No	Ingreso.cod_ingreso ON DELETE CASCADE	Ingreso al que pertenece el renglón	
cantidad	INT		No		Cantidad de botellas	CHECK (cantidad > 0) (RN-15)

Tabla: Deposito_central

Depósito central que recibe la producción embotellada y la redistribuye a las salas. Existe un único depósito.

Campo	Tipo	Clave	Nulo	Referencia	Descripción	Restricciones
id_deposito	INT	PK	No		ID único del depósito	Autoincremental (IDENTITY)
nombre	VARCHAR(50)		No		Nombre del depósito	

Tabla: Stock_Deposito

Stock de cada vino en el depósito central.

Campo	Tipo	Clave	Nulo	Referencia	Descripción	Restricciones
cod_vino	INT	PK, FK	No	Vino.cod_vino ON DELETE RESTRICT	Vino	
id_deposito	INT	PK, FK	No	Deposito_central.id_deposito ON DELETE RESTRICT	Depósito	
cantidad	INT		No		Botellas disponibles en el depósito	DEFAULT 0 CHECK (cantidad >= 0) (RN-15)

Tabla: Sala_Venta_Vino

Stock de cada vino en cada sala de venta.

Campo	Tipo	Clave	Nulo	Referencia	Descripción	Restricciones
cod_vino	INT	PK, FK	No	Vino.cod_vino ON DELETE RESTRICT	Vino	
id_sala	INT	PK, FK	No	Sala_Venta.id_sala ON DELETE RESTRICT	Sala de venta	
cantidad	INT		No		Botellas disponibles en la sala	DEFAULT 0 CHECK (cantidad >= 0) (RN-09)

Tabla: Transferencia

Pedido de vinos de una sala al depósito central. Estados: en_confeccion → terminado → procesado (RN-11).

Campo	Tipo	Clave	Nulo	Referencia	Descripción	Restricciones
nro_solicitud	INT	PK	No		Número de solicitud	Autoincremental (IDENTITY)
estado	estado_documento		No		Estado de la transferencia: en_confeccion, terminado o procesado	DEFAULT 'en_confeccion' Transiciones controladas por trigger (RN-11)
fecha	DATE		No		Fecha de confección	DEFAULT CURRENT_DATE Trigger fuerza la fecha actual y prohíbe modificarla (RN-11)
id_sala	INT	FK	No	Sala_Venta.id_sala ON DELETE RESTRICT	Sala solicitante	

Tabla: Transferencia_vino

Renglones de una transferencia: vinos y cantidad. Solo se modifican si está en confección (RN-12).

Campo	Tipo	Clave	Nulo	Referencia	Descripción	Restricciones
nro_solicitud	INT	PK, FK	No	Transferencia.nro_solicitud ON DELETE CASCADE	Transferencia a la que pertenece el renglón	

Campo	Tipo	Clave	Nulo	Referencia	Descripción	Restricciones
cod_vino	INT	PK, FK	No	Vino.cod_vino ON DELETE RESTRICT	Vino solicitado	
cantidad	INT		No		Cantidad de botellas solicitadas	CHECK (cantidad > 0) (RN-15)

Tabla: Auditoria

Registro de cada cambio de precio de un vino. La completa un trigger AFTER UPDATE (RN-13).

Campo	Tipo	Clave	Nulo	Referencia	Descripción	Restricciones
id_auditoria	INT	PK	No		ID único del registro	Autoincremental (IDENTITY)
usuario	VARCHAR(50)		No		Usuario de la base que hizo el cambio	DEFAULT current_user
precio_anterior	NUMERIC(10, 2)		No		Precio antes del cambio	CHECK (precio_anterior > 0)
precio_nuevo	NUMERIC(10, 2)		No		Precio después del cambio	CHECK (precio_nuevo > 0)
fecha_hora	TIMESTAMP		No		Fecha y hora del cambio	DEFAULT now()
cod_vino	INT	FK	No	Vino.cod_vino ON DELETE RESTRICT	Vino cuyo precio cambió	

3. Integridad y lógica de la base

La decisión central del trabajo fue que las reglas de negocio se escriben en la propia base y no en la aplicación. Una regla que vive en PostgreSQL vale para cualquier cliente que se conecte, sea la API, una consola psql o el script de carga, y no puede saltearse por un error en el código de la aplicación. Por eso la API y el front no calculan totales, descuentos ni stock: insertan los datos y muestran lo que la base devuelve.

Para decidir dónde va cada regla usamos un criterio simple:

- Si la regla se resuelve con la estructura del modelo, va en una clave primaria o foránea.
- Si mira una sola fila, va en un CHECK o un UNIQUE.
- Si necesita consultar otras filas u otras tablas, como el stock, los estados de un documento o la suma de un corte, va en un trigger.

- Los cálculos que se reutilizan son funciones, y las operaciones de varios pasos que tienen que ejecutarse juntas son procedimientos.

Objetos implementados

Los archivos de migración 002 a 007 superan todos los mínimos del enunciado:

- UNIQUE además de las claves primarias: 8 (mínimo 4).
- CHECK: 43, cuatro de ellos entre columnas (mínimo 8, uno entre columnas).
- DEFAULT: 12 (mínimo 3).
- Tipos propios: el ENUM estado_documento y el DOMAIN cuit_dom (mínimo uno de cada uno).
- Funciones: fn_descuento_cliente, fn_total_comprobante, fn_stock_sala y fn_antiguedad_anios (mínimo 4).
- Procedimientos: sp_procesar_ingreso y sp_procesar_transferencia (mínimo 2).
- Triggers: 10, con variantes BEFORE, AFTER y diferido (mínimo 6).
- Vistas: v_stock_total, v_ventas_mensuales y v_documentos_pendientes (mínimo 3).
- Roles: el propietario, bodegas_api para la API y bodegas_report de solo lectura, con los permisos de PUBLIC revocados (mínimo 3).

Trazabilidad: regla, mecanismo y prueba

Cada regla de negocio está protegida por un mecanismo de la base y tiene un caso de prueba que intenta violarla:

- RN-01. Precio único por vino y mayor que cero. Mecanismo: el precio es un atributo de Vino, con un CHECK. Prueba: un vino con precio 0 es rechazado.
- RN-02. Una sala pertenece a una sola localidad y cada localidad tiene un único encargado. Mecanismo: clave foránea más UNIQUE en Encargado. Prueba: un segundo encargado en la misma localidad es rechazado.
- RN-03. Cada empleado trabaja en una sola sala. Mecanismo: clave foránea obligatoria (NOT NULL). Prueba: un empleado con una sala inexistente es rechazado.
- RN-04. Una parcela tiene una sola cepa y se cosecha como máximo una vez por campaña. Mecanismo: clave foránea más UNIQUE sobre finca, parcela y campaña. Prueba: una segunda cosecha en la misma campaña es rechazada.
- RN-05. Los porcentajes del corte de un vino suman 100. Mecanismo: constraint trigger diferido, que se evalúa al confirmar porque mientras se cargan las cepas la suma todavía no es 100. Prueba: una composición del 60 % es rechazada al confirmar.
- RN-06. Descuento según el tipo de cliente: socio 10 %, distribuidor 20 % y exportador 30 %. Mecanismo: el porcentaje se guarda en cada subtipo y lo lee fn_descuento_cliente. Prueba: la función devuelve 10, 20 y 30 según el tipo.
- RN-07. El vencimiento de un permiso de exportación es posterior a su emisión. Mecanismo: CHECK entre columnas. Prueba: un vencimiento anterior a la emisión es rechazado.
- RN-08. El total del comprobante lo calcula la base. Mecanismo: fn_total_comprobante y los triggers de venta. Prueba: si se cargan importes a mano, quedan los calculados.

- RN-09. No se vende más de lo que hay en la sala. Mecanismo: trigger BEFORE con bloqueo de la fila de stock (FOR UPDATE) más un CHECK de cantidad no negativa. Prueba: vender 100 botellas con stock 10 es rechazado con el mensaje «Stock insuficiente».
- RN-10. Solo un ingreso terminado impacta en el depósito y los estados no retroceden. Mecanismo: ENUM, trigger de estados y procedimiento de procesamiento. Prueba: volver un ingreso a «en confección» es rechazado.
- RN-11. Lo mismo para las transferencias, cuya fecha es la actual y no se modifica. Mecanismo: triggers de estado y de fecha y procedimiento de procesamiento. Prueba: si se envía una fecha del año 2000, queda guardada la fecha de hoy.
- RN-12. Un documento que ya no está en confección no admite cambios en sus renglones. Mecanismo: trigger sobre los renglones. Prueba: agregar un renglón a una transferencia terminada es rechazado.
- RN-13. Todo cambio de precio queda auditado. Mecanismo: trigger AFTER UPDATE que escribe en Auditoria. Prueba: subir el precio de un vino agrega una fila en Auditoria.
- RN-14. No se borra un transportista, una bodega ni un vino con movimientos. Mecanismo: claves foráneas con ON DELETE RESTRICT. Prueba: borrar un vino vendido es rechazado.
- RN-15. Cantidades, superficies, kilos y precios positivos, y graduación entre 5 y 20. Mecanismo: CHECK. Prueba: una graduación de 21 y kilos negativos son rechazados.

Pruebas

Las pruebas de las reglas están en tests/db_tests.sql. Corren dentro de una transacción que se revierte al final, así que no modifican la base, e informan PASS por cada regla. El resultado verificado es 15 de 15, tanto en la base vacía como con la carga completa.

Los privilegios de cada rol se prueban aparte en tests/db_tests_roles.sql, con 16 de 16 casos correctos. Entre otras cosas verifican que la API no puede leer la auditoría, crear ni borrar tablas ni deshabilitar triggers, y que el rol de reportes solo puede leer.

Los triggers rechazan las reglas de negocio con RAISE EXCEPTION y el código SQLSTATE P0001. Así la API puede distinguirlas de los errores de constraints y devolverlas como 409 con el mensaje de la regla.

Concurrencia

Demostramos con dos sesiones simultáneas el caso de dos ventas que compiten por la última botella de un vino en una sala. Las capturas de cada escenario están en docs/etapa2_concurrencia/capturas.

- Sin bloqueo, las dos sesiones leen stock 1 y las dos venden: se vende dos veces la misma botella.
- Con SELECT … FOR UPDATE, la segunda sesión espera a que la primera confirme y después lee 0, así que no vende.
- Con el trigger real del sistema, la segunda sesión espera y recibe el error «Stock insuficiente».

- Con el nivel REPEATABLE READ en lugar del bloqueo, la segunda sesión falla por un error de serialización.

Elegimos el nivel READ COMMITTED, que es el predeterminado de PostgreSQL, con FOR UPDATE dentro del trigger de venta. Las razones fueron tres: no obliga a la aplicación a reintentar la transacción, el error le explica al usuario el motivo real del rechazo, y la protección está en la base, por lo que vale para cualquier cliente.

---

4. Carga masiva

La carga se hace con un script en Python, db/seed/generar.py, que se ejecuta con un solo comando sobre la base recién creada:

```
docker compose run --rm seed
```

Catálogos y datos generados

Los catálogos que representan datos reales se leen desde archivos CSV en db/seed/catalogos/: localidades, departamentos, cepas, maridajes, presentaciones y tipos de vino. El resto (bodegas, fincas, clientes, movimientos y ventas) se genera con la biblioteca Faker. La carga es reproducible por dos razones:

- usa una semilla fija, así que dos ejecuciones producen los mismos datos;
- las fechas se calculan relativas al día de la carga, porque las consultas de la sección 8 usan CURRENT_DATE.

Volumen

Tabla	Filas
Renglón de comprobante	600.000
Comprobante	150.000
Cliente	20.000
Permiso de exportación	~20.000
Lote y análisis de calidad	5.000 cada uno
Ingreso y transferencia (con sus renglones)	5.000 cada uno
Parcela	1.000

Vino	500

Orden de carga

El orden respeta las dependencias entre claves foráneas:

1. Primero los catálogos (localidades, cepas, maridajes).
2. Después las bodegas, las salas y sus empleados y encargados.
3. Siguen la producción (fincas, parcelas, lotes y análisis), los transportistas, los vinos con su composición y maridajes, y los clientes con sus subtipos.
4. Al final, el depósito con su stock, los ingresos, las transferencias y los comprobantes con sus renglones.

Estrategia frente a los triggers

Insertar 600.000 renglones uno por uno con los triggers activos tardaría demasiado. Por eso, durante la carga histórica se deshabilitan únicamente los triggers de usuario de las tablas de movimientos. Las claves foráneas y los CHECK siguen activos todo el tiempo. Un bloque finally vuelve a habilitar los triggers aunque la carga falle a mitad de camino. Como deshabilitar triggers requiere ser dueño de las tablas, la carga se ejecuta con el rol propietario y nunca con el de la API.

Esa decisión tiene una consecuencia: como los triggers no recalculan los datos históricos, el generador tiene que hacer exactamente los mismos cálculos que haría la base:

- el precio del renglón es el precio del vino;
- el descuento es el que corresponde al tipo de cliente;
- `el subtotal es cantidad × precio × (1 − descuento);`
- el total del comprobante es la suma de sus renglones.

Tiempos

La inserción se hace con el comando COPY de PostgreSQL en lugar de insertar fila por fila. Medido con db/seed/medir_carga.py, COPY resultó 28 veces más rápido: 0,067 s contra 1,895 s para 10.000 filas. La carga completa por defecto tarda entre 22 y 27 segundos, medido sobre un clon limpio del repositorio.

Consultas de control

Después de la carga se ejecuta db/seed/controles.sql, que verifica que el generador haya respetado las mismas reglas que la base. Los siete controles dan cero inconsistencias:

- vinos sin bodega;
- stock negativo en depósito;
- stock negativo en salas;
- renglones sin comprobante;
- totales de comprobante incorrectos;

- descuentos distintos al tipo de cliente;
- subtotales de renglón incorrectos.

Datos pensados para las consultas

Para que las consultas de la sección 8 tengan resultados con sentido, el generador incluye a propósito ciertos casos:

- ventas de la última semana y de más de cinco años;
- vinos que se venden mucho más que otros;
- vinos que solo se venden en algunas salas;
- clientes que nunca compraron;
- una bodega pequeña con solo tres vinos;
- una cepa del catálogo que no se usa;
- documentos en los tres estados.

En la nube se cargó una escala reducida (30.000 comprobantes y 120.000 renglones) por el límite de 1 GB del plan gratuito de Neon. Se usó el mismo script y la misma semilla, y los controles también dieron cero.

5. Índices y rendimiento

Elegimos seis consultas candidatas de la sección 8 del enunciado y medimos cada una con EXPLAIN (ANALYZE, BUFFERS) antes y después de crear el índice propuesto, sobre una base con 150.000 comprobantes y 600.000 renglones, ejecutando ANALYZE antes de cada medición. Quedaron cinco índices en db/migrations/008_indices.sql. Las consultas S8, S3 y S12 mejoraron, hasta 14 veces en el mejor caso, y los índices pensados para S9 y S10 se descartaron porque no aportaban mejora.

Las mejoras más claras vinieron de tres ideas. Primero, los índices con INCLUDE guardan en el índice las columnas que la consulta necesita, de modo que PostgreSQL responde sin leer la tabla (S8 pasó de 1.246 a 28 buffers). Segundo, en un índice compuesto el orden de las columnas importa: la igualdad va primero y el rango después (id_sala, fecha), lo que llevó a S3 de 6,99 a 0,48 ms. Tercero, PostgreSQL no crea índices sobre las claves foráneas por sí solo, así que buscar renglones por vino obligaba a recorrer toda la tabla hasta que creamos idx_detalle_vino (S12 (a) pasó a ser 4,4 veces más rápida).

En otros dos casos el índice no ayudó. En S9, un índice sobre nombre_comercial no aportó nada porque la restricción UNIQUE (nombre_comercial, anada, volumen_ml) ya genera un índice que sirve para esa búsqueda. En S10 el índice existía pero el planificador no lo usó, porque la consulta se resuelve sobre una CTE y no sobre la tabla lote, y el tiempo se mantuvo en unos 930 ms. Un índice solo sirve si el plan puede aprovecharlo, y eso se comprueba mirando el plan de ejecución.

Los índices también tienen un costo. Al cargar los 150.000 comprobantes con sus dos índices el tiempo aumentó 81 % (de 402 a 729 ms) y al cargar los 600.000 renglones con su índice

aumentó 60 % (de 1.317 a 2.102 ms), es decir, entre 1 y 2 microsegundos más por fila. Como ese costo es chico frente a la mejora en lectura, mantuvimos los cinco índices.

Tabla 1. Índices creados (008_indices.sql)

Índice	Tipo	Consulta	Justificación
idx_ventas_trimestre comprobante (fecha) INCLUDE (id_sala, total)	Con INCLUDE (Index Only Scan)	S8	Un solo rango sobre fecha; los INCLUDE permiten responder sin leer la tabla (1.246 → 28 buffers).
idx_comprobante_sala_fecha comprobante (id_sala, fecha) INCLUDE (id_cliente)	Compuesto	S3	Igualdad primero (id_sala), rango después (fecha); INCLUDE de id_cliente evita el acceso a la tabla (6,99 → 0,48 ms).
idx_detalle_vino detalle_comprobante (cod_vino) INCLUDE (nro_comprobante)	Sobre clave foránea	S12	PostgreSQL no indexa FKs; la PK de renglones empieza por nro_comprobante y no sirve para buscar por vino (27,8 → 6,31 ms).
idx_transferencia_vino transferencia_vino (cod_vino) INCLUDE (nro_solicitud, cantidad)	Sobre clave foránea	S4	Misma razón que el anterior, sobre los renglones de transferencias.
idx_transferencia_pendientes transferencia (id_sala) WHERE estado <> 'procesado'	Parcial	S5 y vista v_documentos_pendientes	Indexa solo lo pendiente (~30 % más chico); el predicado es constante, sin CURRENT_DATE.

Tabla 2. Comparativa antes / después

Consulta	Índice	Antes	Después	Mejora	Recorrido	Filas de tabla	Buffers
S8 ventas por sala, último trimestre	idx_ventas_trimestre	7,77 ms	1,22 ms	6,4x	Parallel Seq → Index Only	150.000 → 5.487	1.246 → 28
S3 clientes por tipo, sala y semana	idx_comprobante_sala_fecha	6,99 ms	0,48 ms	14,5x	Parallel Seq → Index Only	150.000 → 20	1.331 → 92
S12 (a) compraron todos los vinos de una bodega	idx_detalle_vino	27,8 ms	6,31 ms	4,4x	Parallel Seq → Index Only	600.000 → 528	6.901 → 2.389
S12 (b) nunca compraron vino tinto	idx_detalle_vino	155,5 ms	93,0 ms	1,7x	Seq → Index Only (x120)	600.000 → 141.248	5.908 → 2.238
S9 ficha de un vino	(idx_vino_nombre_comercial)	2,08 ms	2,11 ms	sin cambio	Index Scan (unique_nombre)	1 → 1	93 → 93
S10 rendimiento por parcela	(idx_rendimiento_parcela)	933,7 ms	929,2 ms	sin cambio	Seq Scan sobre lote + CTE	5.000 → 5.000	49 → 49

Tabla 3. Índices evaluados y descartados

Índice	Consulta	Motivo del descarte
idx_vino_nombre_comercial ON vino (nombre_comercial)	S9	Duplica el índice de unique_nombre; el tiempo no cambia (2,08 vs 2,11 ms) y solo suma costo de escritura.

Índice	Consulta	Motivo del descarte
idx_rendimiento_parcela ON lote (id_finca, campania) INCLUDE (kilos, nro_parcela)	S10	Sin cambio (933,7 vs 929,2 ms): la CTE se materializa y el planificador nunca usa el índice sobre lote.

Tabla 4. Costo de escritura

Tabla	Filas	Sin índices	Con índices	Sobrecosto	Extra por fila
comprobante (2 índices)	150.000	402 ms	729 ms	+327 ms (+81 %)	~2,2 µs
detalle_comprobante (1 índice)	600.000	1.317 ms	2.102 ms	+785 ms (+60 %)	~1,3 µs

Tamaño en disco: idx_detalle_vino ocupa 13 MB; los dos índices de comprobante, unos 9,1 MB en total.

6. Consultas SQL

Las consultas de la sección 8 del enunciado están resueltas en cuatro archivos de la carpeta consultas/ del repositorio: update.sql, insert.sql y delete.sql tienen seis consultas cada uno, y select.sql tiene doce. Todas están escritas en SQL de PostgreSQL y comentadas. Cada consulta lleva un encabezado con la consigna, la estrategia usada para resolverla y la cantidad de filas afectadas con la carga por defecto. Los valores concretos (nombres de bodega, sala, localidad o vino) se eligen del dataset cargado y se definen como parámetros al comienzo de cada archivo, así se pueden cambiar sin tocar las consultas. Cuando la consigna habla de «la última semana» o «el último trimestre», el período se calcula a partir de CURRENT_DATE y no con fechas fijas.

Las consultas que modifican datos se ejecutan cada una en su propia transacción, con la misma estructura: se abre la transacción, se ejecuta una consulta de control que muestra el estado previo, luego la sentencia y una segunda consulta de control con el estado posterior, y finalmente ROLLBACK. Así se puede comprobar el efecto de cada una y el archivo se puede correr las veces que se quiera sin alterar la base. Además, las consultas respetan las reglas de la base en lugar de esquivarlas: los cambios de precio quedan registrados en Auditoria, las ventas pasan por los triggers de stock y descuento, y los borrados dependen de las acciones referenciales definidas en el modelo.

A continuación comentamos las más interesantes.

Modificación (UPDATE)

- U1 recalcula los importes de todos los comprobantes según el tipo de cliente. Primero cada renglón toma su descuento con fn_descuento_cliente y recalcula su subtotal; después cada comprobante toma subtotal, descuento y total con fn_total_comprobante. Los tres importes se actualizan juntos para que se cumpla el CHECK que exige total =

subtotal − descuento. Con la carga consistente no cambia ningún importe; la consulta sirve para recalcular los totales después de un cambio de descuento como el de U6.

- U3 incrementa un 13 % el precio de los vinos de una bodega. Afecta 29 vinos, y el trigger de auditoría agrega automáticamente 29 filas en Auditoria (RN-13), sin que la consulta tenga que ocuparse.
- U4 intercambia los días de cierre lunes y miércoles en una sola sentencia, con un UPDATE y un CASE. No hace falta un valor intermedio porque PostgreSQL evalúa cada fila con su valor original.
- U5 baja un 15 % el precio de los diez vinos menos vendidos. Usa una CTE con las botellas vendidas por vino y un LEFT JOIN, para que un vino sin ventas también cuente como poco vendido.

Inserción (INSERT)

- I2 crea un maridaje nuevo solo si existe algún vino tinto con graduación de 14 o más, con INSERT … SELECT … WHERE EXISTS, y lo asocia a esos 76 vinos en la misma transacción.
- I3 crea un comprobante con tres renglones usando las funciones de cálculo. El comprobante se inserta sin importes y cada renglón solo con el vino y la cantidad: los triggers validan y descuentan el stock, completan el precio y el descuento y recalculan el total. La consulta muestra después que el total guardado coincide con fn_total_comprobante.
- I6 crea un ingreso de 120 botellas sin indicar estado ni fecha: los valores por defecto lo dejan en confección y con la fecha del día.

Borrado (DELETE)

- D1 borra los comprobantes de socios con más de cinco años de antigüedad con DELETE … USING. Los 56.184 renglones se borran solos por el ON DELETE CASCADE, y el stock no se devuelve porque se trata de historial.
- D2 elimina las cepas que no componen ningún vino ni están plantadas en ninguna parcela, con un doble NOT EXISTS. Si se intentara borrar una cepa en uso, la clave foránea con ON DELETE RESTRICT lo impediría; el NOT EXISTS evita llegar a ese error.
- D4 y D6 borran socios que nunca compraron. Se borra de Cliente y la fila del subtipo se elimina en cascada.
- D5 elimina los ingresos procesados de hace más de un año. Es un caso interesante porque el trigger que bloquea los renglones de un documento cerrado (RN-12) no impide el borrado en cascada: cuando la cascada se ejecuta, el ingreso ya no existe.

Consultas (SELECT)

- S2 y S12 resuelven divisiones relacionales, es decir, preguntas con la forma «los que cumplen para todos». S2, los vinos vendidos en todas las salas, se resuelve por conteo: un vino se vendió en todas las salas si la cantidad de salas distintas donde se vendió es igual al total de salas. S12, los clientes que compraron todos los vinos de una bodega, usa un doble NOT EXISTS: se seleccionan los clientes para los que no existe un vino de

esa bodega que no hayan comprado. Las dos formas son equivalentes; en S2 se eligió el conteo porque recorre los renglones una sola vez.

- S6 y la segunda parte de S12 son anti-joins. Buscan, respectivamente, los vinos sin stock en una sala pero con stock en el depósito y los clientes que nunca compraron un vino tinto. PostgreSQL resuelve el NOT EXISTS con un Hash Anti Join, en una sola pasada.
- S10 calcula el rendimiento en kilos por hectárea de cada parcela y campaña y marca las parcelas que rinden menos que el promedio de su finca. Usa una CTE y una subconsulta correlacionada que compara cada parcela solo con las de su misma finca y campaña, para no mezclar cosechas de años distintos. Es la consulta más costosa (cerca de 1,5 s), porque la subconsulta se evalúa por cada fila; con AVG(...) OVER (PARTITION BY ...) se obtendría lo mismo en una sola pasada.
- S11 usa funciones de ventana. RANK() OVER (PARTITION BY sala) arma el ranking de vinos dentro de cada sala, y SUM(...) OVER (PARTITION BY sala, año ORDER BY mes) calcula las ventas acumuladas mes a mes.

Para las consultas avanzadas (S9 a S12), el mismo archivo incluye su EXPLAIN ANALYZE con los tiempos reales sobre la carga completa: S9 tarda unos 4 ms, S10 cerca de 1,5 s, S11 unos 200 ms y S12 unos 190 ms. Esos planes sirvieron de punto de partida para elegir los índices de la sección 5.

7. API y front

Arquitectura

El sistema tiene tres capas. El navegador ejecuta un front en React que no hace ningún cálculo, el backend es una API REST hecha con FastAPI, y detrás está PostgreSQL, donde viven todas las reglas de negocio. Todas las llamadas del front pasan por un único módulo (front/src/api.js), cuya URL base se configura por variable de entorno, así que ninguna pantalla arma pedidos por su cuenta.

Todo el SQL está escrito a mano con el driver nativo psycopg 3 y no usamos ORM. Cada consulta vive en api/repositories.py, un módulo con una función por consulta. De esa forma el SQL queda a la vista y no se esconde en los controladores, que es lo que pide el enunciado.

Acceso a datos y seguridad

El acceso a datos se apoya en dos reglas que respetamos en todo el código. La primera es que ningún valor que llega del usuario se concatena al texto del SQL: todos los parámetros se pasan con placeholders (%s) de psycopg. La segunda es que cualquier operación de más de una sentencia corre dentro de una transacción explícita.

El caso más importante es la venta. POST /comprobantes inserta la cabecera y todos los renglones en una sola transacción. Por cada renglón, el trigger trg_detalle_validar completa el precio, el descuento del cliente y el subtotal, y el trigger de stock valida la cantidad. Si falla un

solo renglón se hace rollback de todo y la API devuelve el error sin dejar filas a medias. La API no recalcula totales, descuentos ni stock; eso lo resuelve siempre la base.

La API no se conecta como propietario. Usa el rol bodegas_api, creado en la etapa 2, que puede leer y escribir en los esquemas que necesita pero no tiene permisos de DDL. Las credenciales llegan por variables de entorno. El repositorio incluye un .env.example con las variables documentadas y sin valores reales, y el .env de desarrollo está fuera del control de versiones, algo que revisamos en el historial del proyecto.

Endpoints

Los endpoints cubren la tabla de mínimos del enunciado.

Recurso	Endpoints	Comentario
Vinos	GET /vinos, GET /vinos/{cod}	Filtros por tipo, cepa, bodega y añada, búsqueda por texto y paginación. El detalle incluye composición y maridajes.
Vinos	POST, PUT, DELETE /vinos	Alta con composición. La modificación audita el cambio de precio en la tabla Auditoria. La baja es rechazada por la base si el vino tiene renglones vendidos.
Clientes	CRUD sobre /clientes	Contempla los tres tipos (socio, distribuidor y exportador) sobre el mismo supertipo.
Comprobantes	POST /comprobantes, GET /comprobantes/{nro}	Venta completa en una sola transacción.
Ingresos y transferencias	POST (crear), POST /renglones, POST /terminar, POST /procesar	Siguen el circuito de estados del enunciado. Los renglones solo se aceptan mientras el documento está en confección. /procesar llama directamente a sp_procesar_ingreso o sp_procesar_transferencia.

Reportes	/reportes/top-vinos, /ventas-por-sala, /vinos-sin-stock, /transportista-activo, /trazabilidad/{cod}	Solo lectura.
Salud	GET /salud	Verifica la conexión con la base e informa con qué usuario se conecta.

La documentación OpenAPI/Swagger que genera FastAPI está disponible en /docs y coincide con las rutas reales del servicio.

Traducción de errores

Los errores de PostgreSQL se traducen a códigos HTTP en un manejador central (api/errors.py), a partir del SQLSTATE que informa el driver.

Situación en PostgreSQL	SQLSTATE	HTTP
Regla de negocio rechazada (RAISE EXCEPTION)	P0001	409, con el mensaje de la regla
UNIQUE o FOREIGN KEY violados	23505 / 23503	409
CHECK o NOT NULL violados	23514 / 23502	422
Dato no encontrado dentro de un procedimiento	P0002	404
Formato de texto inválido para el tipo	22P02	400

Fallo de serialización o deadlock	40001 / 40P01	503, con sugerencia de reintentar

Cuando una regla de negocio rechaza una operación, el usuario ve el mensaje tal como lo escribió la base, por ejemplo: "Stock insuficiente del vino Los horneros (codigo 308) en la sala 6: hay 0, se piden 5 (RN-09)". El front no necesita conocer la base; muestra el mensaje y el SQLSTATE exactamente cómo llegan.

Front

El front tiene las cinco pantallas mínimas que pide el enunciado.

El catálogo de vinos es un listado paginado con filtros, y su detalle muestra la composición y los maridajes. La pantalla de alta y edición de vino incluye el formulario de composición de cepas y muestra el rechazo cuando el corte no suma 100.

En la pantalla de venta se eligen la sala y el cliente, se agregan los renglones y el total aparece tal como lo devuelve la base. El botón de confirmar se habilita solo cuando hay sala, cliente y renglones cargados, y si falta algo la propia página lo indica.

La pantalla de transferencias permite crearlas, cargar renglones y cambiar el estado con botones que se habilitan según el estado actual del documento. Por último, el tablero muestra el ranking de los diez vinos más vendidos, un gráfico de ventas por sala del último trimestre y los vinos sin stock de una sala elegida.

Pruebas automáticas

Escribimos 18 pruebas en api/tests/test_api.py, bastante por encima del mínimo de 8. Una parte cubre el uso normal (salud, paginación, filtros, códigos 404 y 422). La otra provoca a propósito violaciones de reglas de la base y verifica el código HTTP: una venta con stock insuficiente, un renglón agregado a una transferencia ya terminada, una composición de cepas que no suma 100 y el procesado de una transferencia que todavía no fue terminada. En los cuatro casos la API responde 409 con el mensaje de la regla.

Las pruebas no dependen de una carga de datos puntual, porque buscan los registros que necesitan a través de la propia API. Por eso pasan tanto contra la base local (docker compose up levanta base, API y front en una máquina limpia) como contra la base en la nube.

Demostración de inyección SQL fallida

La evidencia está en docs/evidencia_inyeccion_sql.txt. Sobre la URL pública mandamos a un endpoint con filtro el payload clásico de consulta tautológica: GET /api/vinos?q=' OR '1'='1. El resultado fue 0 vinos. El texto completo llega a la base como un valor de binding y se busca literalmente en el nombre del vino. Si el SQL se hubiera armado por concatenación, la

condición agregada habría dejado el WHERE siempre verdadero y la consulta habría devuelto todo el catálogo.

Los payloads con instrucciones destructivas, del tipo DROP TABLE, son interceptados antes de llegar a la API cuando se envían a la URL pública. Para esos casos hicimos la demostración contra la API local levantada con Docker. En ambos entornos el resultado se explica de la misma manera: ninguna consulta arma el SQL con texto del usuario.

8. Despliegue en la nube

Elección de proveedores

Combinamos dos proveedores gratuitos, cada uno para lo que mejor resuelve.

La base de datos está en Neon (PostgreSQL 18, región São Paulo, aws-sa-east-1), que es la más cercana a nosotros. Lo elegimos porque su plan gratuito alcanza para el volumen reducido que cargamos, porque es PostgreSQL real y sin capas propietarias (todo el SQL del proyecto corre igual en local y en la nube), porque exige TLS en todas las conexiones y porque ofrece branches y snapshots que facilitan la restauración. Descartamos Supabase porque en su plan gratuito la base se suspende por inactividad, y la base gratuita de Render porque vence a los 30 días y se perderían los datos.

La API y el front están en Render. Neon no era opción para alojar la API, ya que sus funciones solo ejecutan JavaScript y TypeScript y la nuestra está en Python. Render construye y ejecuta una imagen Docker que contiene la API y el front ya compilado, y los publica bajo una única URL con HTTPS: https://bodegas-atuel.onrender.com. Que la API sirva el front desde la misma raíz fue una decisión deliberada, porque evita el problema de CORS y deja el despliegue en un solo artefacto. El archivo render.yaml es el Blueprint que declara el servicio (apunta a Dockerfile.deploy y lista las variables de entorno necesarias), y cada push a la rama vigilada dispara el build automáticamente.

Configuración de seguridad

La seguridad se apoya en cuatro medidas.

La primera es la conexión cifrada. El DSN de la API exige sslmode=require y channel_binding=require, de modo que el servidor rechaza cualquier conexión que no use TLS.

La segunda es el mínimo privilegio. La API se conecta con el rol bodegas_api y nunca con el propietario de la base, y el endpoint /salud lo comprueba respondiendo "user": "bodegas_api".

La tercera es que los secretos no pasan por el repositorio. El DSN y las claves están en las variables de entorno del panel de Render, y revisamos el historial de git sin encontrar claves versionadas.

La cuarta es el control de acceso a la base. El enunciado recomienda restringirlo por IP o red interna, pero el plan gratuito de Neon no ofrece listas de IPs permitidas. Por eso el acceso queda protegido por TLS obligatorio y por las credenciales del rol. Lo tomamos como una limitación aceptada del plan, y la restricción por IP queda planteada como mejora si el proyecto pasara a un plan pago.

Migraciones y datos

La base de la nube se creó desde cero con los archivos del repositorio. Aplicamos las migraciones 001 a 008 en orden sobre una base vacía (esquema físico, funciones, procedimientos, triggers, vistas, roles e índices), de modo que cualquier persona puede reproducir el despliegue completo.

Sobre esa base cargamos una escala reducida de datos con el generador de la etapa 3, usando la misma semilla: alrededor de 30.000 comprobantes y 120.000 renglones, muy por encima del mínimo de 10.000 filas en una tabla que pide el enunciado. La reducción se debe al límite de 1 GB de almacenamiento del plan gratuito de Neon; el volumen completo se mantiene en la versión local. Las consultas de control de integridad de la carga se ejecutaron en la nube y dieron cero inconsistencias.

Backup y restauración

Probamos el circuito completo. Tomamos un pg_dump en formato custom (-Fc) de la base de Neon y lo restauramos con pg_restore --no-acl --no-owner en una base nueva, un contenedor con PostgreSQL 18 equivalente al de la nube. Para verificarlo comparamos los conteos de filas de las 32 tablas antes y después, y coinciden exactamente (evidencia en docs/etapa8_despliegue/evidencia/05_backup_restore.txt). Con esto el proyecto se puede borrar y reconstruir por completo a partir del repositorio y de su backup.

Una regla de la base funcionando en la nube

Para cumplir la condición del enunciado comprobamos la regla RN-09 por HTTP contra la URL pública. Un POST /api/comprobantes que pide más botellas de las que hay en stock devuelve un 409 Conflict con el mensaje de la regla y su SQLSTATE:

{"detail": "Stock insuficiente del vino Los horneros (codigo 308) en la sala 6: hay 0, se piden 5 (RN-09)", "sqlstate": "P0001"}

La transacción hace rollback y la base queda íntegra (evidencia en docs/etapa8_despliegue/evidencia/07_url_publica.txt). También comprobamos que la URL pública funciona desde dispositivos externos.

Limitaciones del plan gratuito

Render apaga el servicio después de unos 15 minutos sin uso, y el primer pedido posterior tarda entre 30 y 60 segundos (cold start). Neon, por su parte, aplica scale-to-zero al compute

tras 5 minutos de inactividad, así que la primera consulta tiene que despertar la base. Ninguno de los dos comportamientos es un error del sistema. Para una demostración alcanza con abrir la aplicación unos minutos antes.

9. Conclusiones

El trabajo nos llevó de un enunciado de negocio a un sistema completo que funciona: el modelo de datos, su implementación en PostgreSQL con las reglas protegidas en la propia base, una carga de 600.000 renglones reproducible, índices justificados con planes de ejecución, una API REST, un front en React y el despliegue en la nube. Lo más valioso fue ver que cada decisión del modelo de la primera etapa tenía consecuencias concretas en todas las siguientes.

Dificultades

La primera dificultad fue decidir dónde vive cada regla. Al principio muchas parecían candidatas a resolverse en la API, y tuvimos que aprender a distinguir:

- una restricción sobre una sola fila es un CHECK;
- una relación es una clave foránea;
- lo que depende de otras filas o tablas, como el stock, los estados o la suma del corte, necesita un trigger.

La segunda fue la carga masiva. Para cargar 600.000 renglones en pocos segundos hubo que deshabilitar los triggers durante la carga histórica, y eso obligó a que el generador replicara exactamente los cálculos de la base. Las consultas de control fueron las que nos dieron la certeza de que los datos eran coherentes.

También aprendimos que los datos de prueba tienen que pensarse junto con las consultas. Varias consultas de la sección 8 devolvían cero filas hasta que el generador incluyó a propósito los casos que las hacían interesantes, como clientes que nunca compraron o una bodega con pocos vinos.

Por último, la concurrencia: ver en dos terminales cómo se vendía dos veces la misma botella sin bloqueo nos hizo entender por qué el trigger de venta usa FOR UPDATE.

Qué cambiaríamos

Definiríamos desde la primera etapa la convención de errores de los triggers y su traducción a códigos HTTP, en lugar de ajustarla cuando llegamos a la API. Escribiríamos las migraciones pensando desde el comienzo en que se ejecutan en más de un entorno (local y nube). Y con más tiempo sumaríamos al front una pantalla de ingresos al depósito y pasaríamos el despliegue a un plan pago, para evitar la demora del primer acceso después de un rato sin uso.

Además, al momento de hacer los índices, observamos que sería más conveniente tener todas las consultas preparadas de antemano, para ahorrar tiempo y mejorar la eficiencia.

Aprendizajes

A lo largo del desarrollo del proyecto, pudimos reafirmar o aprender los siguientes conceptos:

- **Las reglas en la base valen para todos.** Una regla escrita en la base vale para cualquier cliente que se conecte, sea la API, psql o el script de carga. Por eso la API y el front pueden limitarse a mostrar lo que la base calcula.
- **Un índice se justifica con evidencia.** No basta con suponer que va a servir: dos de los candidatos que evaluamos no mejoraban nada y quedaron descartados después de mirar el plan de ejecución.
- **El trabajo en equipo con ramas y revisiones funciona.** Trabajar con una rama por tarea y revisión entre los tres antes de integrar nos permitió avanzar en paralelo sobre etapas distintas sin pisarnos.